

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

МОСКОВСКИЙ ИНСТИТУТ ЭЛЕКТРОНИКИ И МАТЕМАТИКИ
им. А.Н. Тихонова НИУ ВШЭ

ДЕПАРТАМЕНТ КОМПЬЮТЕРНОЙ ИНЖЕНЕРИИ

КУРСОВАЯ РАБОТА
по дисциплине «Алгоритмизация и программирование»

Тема: «Разработка приложения для пакетной конвертации видеороликов в
эффективный формат хранения»

Выполнил:
студент группы БИВ253
Солодилов Михаил Анатольевич

Руководитель:
Волков Владимир Олегович

Москва, 2026

Задание на курсовую работу

Тема: реализовать программу для пакетной конвертации видеороликов в более эффективный формат для сокращения занимаемого места на диске.

Цель работы: разработать настольное приложение Floppy, позволяющее пользователю выбрать один или несколько видеофайлов и перекодировать их в формат HEVC с настройкой качества, разрешения и частоты кадров. Итоговая программа должна создавать выходные файлы с параметрами, заданными пользователем в интерфейсе.

Задачи работы:

- 1) изучить предметную область хранения и перекодирования видеороликов;
- 2) рассмотреть существующие инструменты и подходы к сжатию видео;
- 3) выбрать технологический стек для реализации настольного приложения;
- 4) реализовать пользовательский интерфейс для выбора файлов и настройки параметров перекодирования;
- 5) реализовать модуль перекодирования через FFmpeg;
- 6) добавить пакетную обработку файлов и отображение прогресса;
- 7) провести тестирование основных сценариев работы программы;
- 8) подготовить пользовательскую и разработческую документацию.

Функциональные требования:

- 1) программа должна позволять выбрать один или несколько видеофайлов через диалог выбора файлов;
- 2) программа должна поддерживать добавление файлов перетаскиванием в окно приложения;
- 3) программа должна принимать на вход локальные видеофайлы, доступные файловой системе пользователя;
- 4) программа должна позволять указать параметр качества перекодирования;

- 5) программа должна позволять ограничить высоту выходного видео;
- 6) программа должна позволять ограничить частоту кадров выходного видео;
- 7) программа должна создавать выходной файл с суффиксом `_compressed`;
- 8) программа должна позволять включить копирование метаданных исходного файла в результат;
- 9) программа должна отображать прогресс выполнения операции;
- 10) программа должна сообщать пользователю об ошибке, если перекодирование завершилось неуспешно.

Нефункциональные требования:

- 1) язык реализации: Python;
- 2) графический интерфейс: GTK 4 через PyGObject;
- 3) обработка видео: FFmpeg и библиотека python-ffmpeg;
- 4) копирование метаданных: FFmpeg и ExifTool;
- 5) платформа: настольная операционная система Linux с установленными FFmpeg и GTK 4;
- 6) формат хранения данных: локальные файлы пользователя, база данных не используется.

Ожидаемые результаты:

- 1) рабочее приложение, выполняющее пакетное перекодирование видео;
- 2) исходный код проекта;
- 3) инструкция по запуску и использованию;
- 4) примеры входных данных и результата перекодирования.

График выполнения

№	Этап работы	Дата
1	Обоснование темы, описание предметной области, постановка цели и задач	29.11.2025
2	Короткий обзор аналогов и источников литературы	13.12.2025
3	Подготовка окончательной версии технического задания	20.12.2025
4	Реализация базового модуля перекодирования через FFmpeg	дата уточняется
5	Реализация графического интерфейса и пакетной обработки файлов	дата уточняется
6	Тестирование приложения и подготовка документации	15.05.2026

Аннотация

В курсовой работе рассматривается разработка настольного приложения Floppy для пакетной конвертации видеороликов в формат HEVC. Объектом разработки является программное средство для уменьшения занимаемого видеофайлами места на диске. Цель работы заключается в создании приложения с графическим интерфейсом, которое позволяет выбрать несколько видеофайлов, настроить качество, разрешение и частоту кадров, запустить перекодирование и получить выходные файлы с суффиксом `_compressed`.

В ходе работы были изучены подходы к сжатию видео, выбран стек Python, GTK 4, PyGObject, FFmpeg, ExifTool и python-ffmpeg, реализованы модули пользовательского интерфейса, выбора параметров, определения доступного HEVC-кодека, запуска FFmpeg, копирования метаданных и отображения прогресса. Результатом стала программа, предназначенная для локальной обработки видеофайлов без использования сетевых сервисов и базы данных.

Работа содержит 21 страницу, 1 иллюстрацию, 1 таблицу, 8 использованных источников.

ОГЛАВЛЕНИЕ

1	Введение	7
2	Обзор аналогов и источников	8
3	Описание разработанного приложения	9
4	Архитектура и реализация	10
4.1	Графический интерфейс	10
4.2	Пакетная обработка	10
4.3	Модуль перекодирования	11
4.4	Выбор кодировщика	11
4.5	Копирование метаданных	12
5	Ход разработки	13
6	Тестирование	14
7	Заключение	15
8	Список использованных источников	16
A	Руководство пользователя	17
A.1	Назначение программы	17
A.2	Установка	17
A.3	Запуск	17
A.4	Основной сценарий использования	17
A.5	Типовые ошибки	18
B	Руководство разработчика	19
B.1	Структура проекта	19
B.2	Поток выполнения	19
B.3	Выбор кодировщика	19
B.4	Сборка и запуск при разработке	20
B.5	Возможные направления развития	20

1 Введение

Видеоролики с мобильных устройств, камер и экранных записей могут занимать много места на диске. Если файл используется для архива, пересылки или просмотра без дальнейшего монтажа, пользователь может уменьшить занимаемое место за счет перекодирования с другими параметрами качества, разрешения или частоты кадров.

Одним из способов уменьшения размера видео является перекодирование в видео-кодек, рассчитанный на более высокое сжатие. В данной работе используется HEVC, также известный как H.265: в описании рекомендации ITU-T указано, что стандарт был разработан в ответ на потребность в более высоком сжатии движущихся изображений [1]. Для практической обработки видео выбран FFmpeg: его документация описывает инструмент как универсальный медиаконвертер, который может читать различные входные данные, фильтровать и транскодировать их в разные выходные форматы [2].

Актуальность темы связана с тем, что работа с FFmpeg в командной строке требует выбора кодека, параметров качества, фильтров и выходного файла. В проекте Florpy эти параметры вынесены в графический интерфейс, а выбор доступного HEVC-кодировщика выполняется программно. Поэтому пользователь запускает перекодирование через элементы управления приложения, а не через ручной ввод команды FFmpeg.

Целью курсовой работы является разработка приложения Florpy для пакетной конвертации видеороликов в эффективный формат хранения. Для достижения цели необходимо решить следующие задачи:

- определить требования к приложению;
- изучить инструменты перекодирования видео;
- реализовать запуск FFmpeg из Python-приложения;
- реализовать выбор файлов и параметров обработки;
- обеспечить отображение прогресса и сообщений о результате;
- провести тестирование основных пользовательских сценариев.

2 Обзор аналогов и источников

Для решения задачи уменьшения размера видео существуют как универсальные консольные инструменты, так и графические приложения.

FFmpeg выбран как базовый инструмент обработки видео, потому что его документация описывает поддержку входных файлов, фильтрации и транскодирования в выходные форматы [2]. Для приложения Floppy это означает, что собственная реализация кодирования видео не требуется: программа формирует параметры и запускает FFmpeg.

HandBrake рассмотрен как графический аналог для перекодирования видео. Его документация описывает готовые профили настроек для устройств и общих сценариев, а также аппаратные профили для кодирования через выделенные медиаблоки современных видеоустройств [3]. В HandBrake также есть функция *Passthru common Metadata*, но документация описывает ее как перенос ограниченного набора распространенных полей метаданных [4]. В Floppy реализован другой сценарий: при включении `Copy metadata` приложение копирует метаданные FFmpeg и главы, а затем переносит теги через ExifTool с сохранением групп тегов [5, 6].

Для реализации интерфейса Floppy используется GTK 4 через PyGObject. GTK предоставляет виджеты для окон, кнопок, полей ввода, диалогов выбора файлов и индикаторов прогресса [7]. Для асинхронной работы с FFmpeg выбрана библиотека `python-ffmpeg`, которая позволяет запускать обработку из Python-кода и получать события прогресса [8].

В результате анализа был выбран подход, при котором графический интерфейс отвечает за взаимодействие с пользователем, а обработка видео выполняется через FFmpeg. Такой вариант соответствует задаче курсовой работы: разработать оболочку для пакетного перекодирования, не реализуя видеокодек самостоятельно.

3 Описание разработанного приложения

Разработанное приложение Floppy представляет собой настольную программу с графическим интерфейсом. Оно предназначено для пакетного перекодирования видеороликов в HEVC. Пользователь выбирает один или несколько файлов, задает параметры обработки и запускает кодирование. Для каждого входного файла создается отдельный выходной файл с суффиксом `_compressed`.

Проект состоит из следующих основных файлов:

- `app.py` — модуль графического интерфейса и пользовательских сценариев;
- `engine.py` — модуль запуска перекодирования;
- `utils.py` — вспомогательный модуль для работы с FFmpeg, FFprobe и параметрами кодировщиков;
- `requirements.txt` — список Python-зависимостей проекта.

Пользовательский интерфейс реализован в классе `MainWindow`. Окно содержит название приложения, строку выбранных файлов, кнопку выбора файлов, подсказку о перетаскивании, поля настройки качества, разрешения и частоты кадров, переключатель копирования метаданных, индикатор прогресса, строку статуса и кнопку запуска перекодирования.

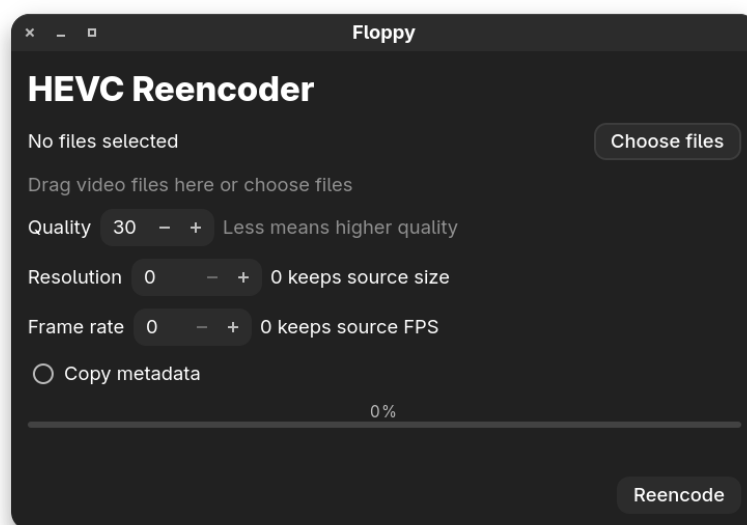


Рис. 1: Главное окно приложения Floppy

4 Архитектура и реализация

4.1 Графический интерфейс

Модуль `app.py` создает GTK-приложение с идентификатором `dev.floppy.Reencoder`. При активации приложения создается главное окно. В интерфейсе используется вертикальное расположение элементов: сверху находится заголовок, далее идут элементы выбора файлов и настройки параметров, затем индикатор прогресса и кнопка запуска.

Выбор файлов реализован через `Gtk.FileChooserNative`. Диалог поддерживает множественный выбор. Дополнительно в окно добавлен обработчик перетаскивания файлов через `Gtk.DropTarget`. Это позволяет пользователю выбрать видео как через стандартный диалог, так и прямым переносом файлов в окно.

Для настройки качества используется числовое поле со значениями от 1 до 51. Разрешение и частота кадров также задаются числовыми полями. Значение 0 для разрешения означает сохранение исходного размера кадра, а значение 0 для частоты кадров означает сохранение исходной частоты кадров. Переключатель `Copy metadata` включает копирование метаданных исходного файла в выходной файл.

4.2 Пакетная обработка

После нажатия кнопки `Reencode` приложение проверяет, выбраны ли файлы. Если список пуст, пользователь получает сообщение `Choose files first`. Если файлы выбраны, интерфейс считывает параметры качества, разрешения, частоты кадров и состояние переключателя копирования метаданных. Затем интерфейс блокирует кнопки выбора и запуска, чтобы пользователь не начал вторую обработку параллельно.

Перекодирование выполняется в отдельном потоке, не блокируя основной поток GTK. Во время обработки пользователи видит полосу прогресса, какой файл перекодировается в текущий момент, сколько файлов уже готово. При возникновении ошибки приложение останавливает перекодировку, разблокирует кнопки и показывает сообщение с количеством уже обработанных файлов.

4.3 Модуль перекодирования

Модуль `engine.py` содержит асинхронную функцию `reencode`. Она принимает путь к входному файлу, параметр качества, ограничение разрешения, ограничение частоты кадров, признак копирования метаданных и необязательную функцию обратного вызова для прогресса.

Перед запуском FFmpeg модуль определяет исходное разрешение и частоту кадров. Если пользовательское ограничение выше или равно исходному значению, параметр не применяется. Это предотвращает бессмысленное увеличение разрешения или частоты кадров при сжатии. Если включено копирование метаданных, модуль заранее проверяет доступность ExifTool, чтобы не запускать длительное перекодирование при отсутствии нужной внешней утилиты.

Имя выходного файла формируется на основе имени входного файла. К исходному имени добавляется суффикс `_compressed`, расширение сохраняется. Далее создается объект FFmpeg, указываются входной файл, выходной файл и рассчитанные параметры кодирования. При включенном копировании метаданных в параметры FFmpeg добавляются `map_metadata=0` и `map_chapters=0`, а после завершения кодирования вызывается ExifTool с аргументами `-TagsFromFile, -all:all` и путем к выходному файлу.

4.4 Выбор кодировщика

Модуль `utils.py` отвечает за выбор доступного HEVC-кодировщика. В проекте описаны варианты `hevc_nvenc`, `hevc_qsv`, `hevc_vaapi`, `hevc_amf`, `hevc_vulkan` и `libx265`. Сначала программа получает список доступных кодировщиков FFmpeg, затем проверяет их в порядке приоритета.

Если доступен аппаратный кодировщик, приложение может использовать его. Если подходящий аппаратный вариант не найден, используется программный `libx265`. Для каждого кодировщика заданы собственные параметры качества, так как разные реализации FFmpeg используют разные имена опций.

Такой подход делает приложение более переносимым между компьютерами с разными видеокартами и конфигурациями FFmpeg. При этом пользователю не нужно вручную выбирать конкретный кодировщик.

4.5 Копирование метаданных

Копирование метаданных реализовано как необязательный режим. Если пользователь включает `Copy metadata`, приложение выполняет две операции. Сначала FFmpeg получает параметры `map_metadata=0` и `map_chapters=0`, которые указывают копировать метаданные и главы из первого входного файла [5]. Затем после успешного перекодирования вызывается ExifTool с командой копирования тегов из исходного файла в выходной. Документация ExifTool описывает форму `-TagsFromFile src -all:all dst` как копирование тегов с сохранением исходных групп [6].

Если ExifTool недоступен, приложение завершает операцию до запуска FFmpeg и возвращает понятную ошибку.

5 Ход разработки

Реализацию проекта можно разделить на несколько логических этапов. Сначала была сформулирована задача: создать программу для пакетной конвертации видеороликов в более эффективный формат. Для реализации выбран стек Python и FFmpeg: Python используется для графического приложения и связующей логики, а FFmpeg — для непосредственной обработки видео.

Затем был реализован базовый модуль перекодирования. Он запускает FFmpeg для одного файла, формирует выходное имя и применяет параметр качества. После этого в проект была добавлена работа с FFprobe для получения данных о видео: количества кадров, разрешения и частоты кадров.

Следующим шагом стала логика выбора кодировщика. Приложение проверяет доступные HEVC-кодировщики и выбирает вариант из заданного списка приоритетов. Это позволяет использовать аппаратное кодирование там, где оно доступно, и программное кодирование через `libx265` как резервный вариант.

После этого был разработан графический интерфейс на GTK 4. Были реализованы выбор файла, настройки качества, разрешения и частоты кадров, шкала прогресса. Затем был добавлен функционал обработки нескольких файлов и их добавление перетаскиванием. В последнюю очередь была сделана работа по копированию метаданных.

6 Тестирование

Для проверки приложения выделены основные контрольные сценарии, связанные с функциональными требованиями.

- 1) **Запуск приложения.** Программа должна открыть главное окно, отобразить элементы управления и не завершиться с ошибкой при наличии необходимых зависимостей.
- 2) **Выбор одного файла.** После выбора одного видео в интерфейсе должно отображаться имя файла, а кнопка запуска должна начинать обработку.
- 3) **Выбор нескольких файлов.** После выбора нескольких файлов приложение должно отображать их количество и последовательно обрабатывать список.
- 4) **Перетаскивание файлов.** Файлы должны добавляться переносом в окно приложения.
- 5) **Настройка качества.** Указанное пользователем значение качества должно передаваться в параметры кодировщика.
- 6) **Ограничение разрешения.** При значении 0 должен сохраняться исходный размер кадра, а при меньшем значении должен применяться фильтр масштабирования.
- 7) **Ограничение частоты кадров.** При значении 0 должна сохраняться исходная частота кадров, а при меньшем значении должен применяться фильтр fps.
- 8) **Копирование метаданных.** При включенном переключателе Copy metadata должны добавляться параметры копирования метаданных FFmpeg и выполняться перенос тегов через ExifTool.
- 9) **Обработка ошибки.** При ошибке FFmpeg приложение должно разблокировать кнопки и вывести сообщение о сбое.

Перечисленные сценарии покрывают основные функции из технического задания: выбор файлов, настройку параметров, запуск перекодирования, копирование метаданных, отображение прогресса, обработку ошибок и сохранение результата в файл с суффиксом `_compressed`.

7 Заключение

В ходе курсовой работы было разработано настольное приложение Floppy для пакетной конвертации видеороликов в эффективный формат хранения. Программа реализована на языке Python с использованием GTK 4 для графического интерфейса и FFmpeg для обработки видео.

В приложении реализованы выбор одного или нескольких файлов, добавление файлов перетаскиванием, настройка качества, ограничение разрешения и частоты кадров, копирование метаданных, последовательное перекодирование файлов, отображение прогресса и сообщений о результате. Для повышения переносимости реализован автоматический выбор доступного HEVC-кодировщика.

Разработанное приложение решает поставленную задачу: позволяет уменьшать размер видеофайлов без ручного ввода команд FFmpeg. В качестве дальнейшего развития можно добавить сохранение пользовательских настроек, журнал выполненных операций и предварительную оценку размера результата.

8 Список использованных источников

- [1] ITU-T. *Recommendation H.265: High efficiency video coding*. URL: <https://www.itu.int/rec/T-REC-H.265> (дата обр. 08.05.2026).
- [2] FFmpeg. *ffmpeg Documentation*. URL: <https://ffmpeg.org/ffmpeg.html> (дата обр. 08.05.2026).
- [3] HandBrake. *Official presets*. URL: <https://handbrake.fr/docs/en/latest/technical/official-presets.html> (дата обр. 08.05.2026).
- [4] HandBrake. *Metadata Passthru*. URL: <https://handbrake.fr/docs/en/latest/advanced/metadata-passthru.html> (дата обр. 09.05.2026).
- [5] FFmpeg. *Metadata options*. URL: <https://ffmpeg.org/ffmpeg-doc.html> (дата обр. 09.05.2026).
- [6] ExifTool. *ExifTool application documentation*. URL: https://exiftool.org/exiftool_pod2.html (дата обр. 09.05.2026).
- [7] PyGObject. *PyGObject Documentation*. URL: <https://pygobject.gnome.org/> (дата обр. 09.05.2026).
- [8] python-ffmpeg. *python-ffmpeg Documentation*. URL: <https://python-ffmpeg.readthedocs.io/> (дата обр. 08.05.2026).

А Руководство пользователя

А.1 Назначение программы

Florpy предназначено для пакетного перекодирования видеороликов в HEVC. Программа помогает уменьшить размер видеофайлов на диске и не требует от пользователя ручного ввода команд FFmpeg.

А.2 Установка

Для работы приложения требуется Linux, Python, GTK 4, FFmpeg, ExifTool и Python-зависимости из файла `requirements.txt`. Перед запуском необходимо установить системные пакеты GTK 4, FFmpeg и ExifTool средствами пакетного менеджера операционной системы, затем установить зависимости Python.

Пример установки Python-зависимостей:

```
pip install -r requirements.txt
```

А.3 Запуск

Запуск приложения выполняется из каталога проекта:

```
python app.py
```

После запуска открывается окно Florpy с элементами выбора файлов и настройки параметров.

А.4 Основной сценарий использования

- 1) нажать кнопку `Choose files` или перетащить видеофайлы в окно приложения;
- 2) выбрать значение качества;
- 3) при необходимости указать максимальную высоту видео;
- 4) при необходимости указать максимальную частоту кадров;
- 5) при необходимости включить `Copy metadata`;

- 6) нажать кнопку Reencode;
- 7) дождаться завершения обработки;
- 8) найти рядом с исходным файлом новый файл с суффиксом `_compressed`.

A.5 Типовые ошибки

Если пользователь нажал Reencode, но не выбрал файлы, программа выводит сообщение `Choose files first`. Нужно выбрать хотя бы один видеофайл и повторить запуск.

Если в системе не установлен FFmpeg или отсутствует подходящий HEVC-кодировщик, перекодирование завершится ошибкой. В этом случае нужно установить FFmpeg или включить кодировщик HEVC в используемой сборке FFmpeg.

Если включено `Copy metadata`, но в системе не установлен ExifTool, приложение сообщит об ошибке до запуска перекодирования. Нужно установить ExifTool или отключить копирование метаданных.

Если выбранный файл поврежден или не является видеофайлом, FFmpeg может завершить обработку с ошибкой. Нужно проверить исходный файл или выбрать другой файл.

В Руководство разработчика

В.1 Структура проекта

Проект Floppy расположен в каталоге `floppy`. Основные файлы:

- `app.py` — точка входа приложения и описание графического интерфейса;
- `engine.py` — логика перекодирования одного файла;
- `utils.py` — работа с FFmpeg, FFprobe, ExifTool, кодировщиками и параметрами вывода;
- `requirements.txt` — список Python-зависимостей.

В.2 Поток выполнения

При запуске `app.py` создается объект `FloppyApp`, затем главное окно `MainWindow`. Пользователь выбирает файлы, после чего метод обработки кнопки считывает значения интерфейса и запускает отдельный поток. В этом потоке для каждого файла вызывается асинхронная функция `engine.reencode`.

Функция `reencode` получает технические данные исходного видео через функции из `utils.py`, формирует параметры FFmpeg и запускает обработку. Если включено копирование метаданных, перед обработкой вызывается проверка ExifTool, а после успешного завершения FFmpeg выполняется перенос тегов в выходной файл. Прогресс передается обратно в интерфейс через `callback`. Обновление GTK-виджетов выполняется через `GLib.idle_add`, чтобы изменения происходили в основном потоке интерфейса.

В.3 Выбор кодировщика

Доступные кодировщики описаны в словаре `ENCODERS`. Порядок выбора задан списком `ENCODER_PRIORITIES`. Для добавления нового кодировщика нужно:

- добавить описание в `ENCODERS`;
- указать имя кодека FFmpeg;

- задать параметры качества;
- при необходимости указать аппаратное ускорение и дополнительные параметры;
- добавить ключ кодировщика в `ENCODER_PRIORITIES`.

В.4 Сборка и запуск при разработке

Перед разработкой нужно установить зависимости:

```
pip install -r requirements.txt
```

Также в системе должны быть доступны исполняемые файлы `ffmpeg`, `ffprobe` и `exiftool`. Проверить их наличие можно командами:

```
ffmpeg -version  
ffprobe -version  
exiftool -ver
```

Запуск приложения для проверки:

```
python app.py
```

В.5 Возможные направления развития

В дальнейшем разработчик может добавить сохранение пользовательских настроек, выбор каталога для результатов, отображение подробного журнала FFmpeg, отмену текущей операции и отдельный режим оценки ожидаемого размера файла.